

Keep Your Instruction, Tests & Solution in Sync

Tasker Notes

Almost every task that gets sent back fails for **one underlying reason**: the **instruction**, the **tests**, and the **reference solution** don't describe the *same contract*. If a requirement lives in one of the three but not the others, the task fails review.

The one rule this whole doc is about:

Everything the instruction asks for is **tested**. Everything the tests enforce is **stated in the instruction**. Your reference solution does **exactly what the instruction says** — no more, no less.

Get those three to agree and you've eliminated the large majority of revision requests. Here's where they most often drift apart, with a self-check for each.

1. Test *everything* the instruction requires — not just the happy path

By far the most common issue. The verifier only checks *part* of what the instruction demands, so a wrong or incomplete solution still passes.

Watch for:

- **Unchecked output fields.** If the instruction requires several fields, test all of them. (e.g., *the instruction requires a `bill_id` on each reversal, but the tests only check the amount and account — so a reversal with a blank `bill_id` passes.*)
- **Hard-codable config.** If a value is the same in every test case, a solution can hardcode it and pass without doing the work. **Vary it across cases.** (e.g., *a tolerance fixed at 50 in every case — hardcoding 50 passes.*)
- **Untested failure modes.** If the instruction describes a rejection/error case, test that it actually happens. (e.g., *every holdout input has zero rejections, so an implementation that never rejects anything still passes.*)
- **Indistinguishable strategies.** Make sure the tests can tell a correct approach from a plausible-but-wrong one. (e.g., *no test case distinguishes "minimize the largest job" from "minimize the number of jobs."*)

Self-check: Write a deliberately wrong or incomplete solution and run it against your tests. Do they fail it? If not, your coverage has a hole.

2. Make your reference solution obey its own contract

Second most common. The oracle quietly does something the instruction doesn't sanction, or skips a requirement the instruction states.

Watch for:

- **Undocumented constraints** the solution adds that the instruction never states. (e.g., *the solution rejects codes longer than 64 characters, but the contract allows any length.*)

- **Silently skipped requirements.** (e.g., the contract says to count malformed rows; the solution drops them without counting.)
- **Edge-case bugs** — integer overflow, non-atomic file replacement, off-by-one at boundaries.

Self-check: Read your instruction as a contract, then read `solve.sh` line by line against it. Does the solution do exactly what the instruction says — nothing extra, nothing missing?

3. Pin down anything that changes the answer

If a choice **materially changes the graded output** but the instruction leaves it open, reasonable solutions diverge — and your grader accepts only one. That's a send-back.

Watch for:

- **Undefined method or model** — which estimator? which distribution? — when grading requires one specific result.
- **Undefined precedence** when two rules both apply — which error wins?
- **Undefined tie-breaks, ordering, or schema** for a required output.

Fix: either *specify the choice*, or *make grading accept every valid interpretation*.

The nuance (important): this is the flip side of "don't over-explain." **Omit** things that are *unambiguous* domain standards a competent practitioner would just know. **Pin** things where competent experts would *legitimately differ* and your grader only accepts one answer. Over-trimming is how tasks become ambiguous.

4. Don't let the verifier enforce unstated rules

If your tests demand something, the instruction must say it — and tolerances must match.

Watch for:

- **Tolerance mismatch.** (e.g., the instruction says results within $\pm 1e-3$, but the verifier requires an exact $1e-9$ match — a correct solution fails.)
- **Brittle exact-golden matches** that bake in incidental artifacts (an off-by-one, a leftover row) that aren't part of the contract.
- **Unstated structural rules** — rejecting a file layout, a duplicate key, or an output shape the docs never prohibit.

Self-check: For every assertion in your tests, point to the sentence in the instruction that requires it. No sentence -> either add it to the instruction or drop the assertion.

Also: don't let the task be gamed

The four areas above keep an *honest* solution from **failing** review. These next two keep a *dishonest* one from **passing** — and integrity holes are one of the **largest** sources of blocking send-backs, not a footnote. Two rules cover it: **the answer stays out of the agent's reach** (§5), and **the reward reflects your test of the agent's output** (§6). An agent works with everything in its environment and will take

any shortcut that scores, so close both paths — it must not be able to read, edit, or infer the answer, and it must not be able to set the reward any way other than by producing output your verifier grades.

5. Keep the ground truth out of the agent's reach

Many tasks let the agent **read, alter, or bypass** the answer rather than compute it. Shut each door:

- **Store answers/goldens in the verifier only.** Expected outputs and ground truth belong in the verifier image (`tests/`) — never in a directory the agent can read (not next to the inputs, not in `/app`).
- **Grade against a pristine, verifier-owned copy.** Don't recompute the "expected" result from files the agent could modify. An agent can swap in an easier input (truncate it, replace it) so your recomputed expectation matches its incomplete output. Compare against a copy the agent never touched.
- **Make held-out cases genuinely independent.** If your "hidden" cases are just re-serialized copies of the visible data, an agent can memorize results by ID and skip building a real solution. Holdouts must require the general answer.
- **Don't leak the label in names or metadata.** Filenames, directory names, and metadata the agent can see must not encode the answer (e.g., a folder literally named for the defect it contains).
- **Watch symlink-following when your verifier copies agent files.** If the verifier stages an agent-controlled directory, copy it *without following symlinks* (or don't stage it as root) — otherwise a planted link can pull protected files into readable space.

***Self-check:** Could a lazy agent pass by reading or editing a file instead of solving the task? If any answer, fixture, or label is reachable or modifiable from the agent's side, close it.*

6. Grade robustly — treat the agent's output as untrusted

Everything the agent produces is adversarial text. Make sure it can't **fake a pass**:

- **Parse defensively.** Don't let agent-controlled strings slip into the fields your grader reads and forge a correct-looking result.
- **Reward real behavior,** not text the agent can shape to look right — execute its program and check outcomes rather than trusting what it reports.
- **Protect the reward channel.** The reward is written **only by the verifier, and only after it grades the output**. Nothing the agent controls may set it: a process the agent started can outlive the run and overwrite the reward file after you've written it, and agent code invoked during grading can write it directly. Keep the reward path out of the agent's reach, and make sure no agent-spawned process is still alive when the reward is written.

***Self-check:** Can the agent fake a pass — by spoofing the text my grader parses, or by writing/altering the reward itself (including via a process it left running)? If so, harden the parsing and lock down the reward path.*

Before you submit — 60-second self-audit

- ☐ Every requirement in the instruction has a corresponding test.
- ☐ A deliberately wrong/incomplete solution **fails** your tests.

- ☐ Tests distinguish a correct approach from a plausible-but-wrong one — no two strategies produce identical graded output.
- ☐ No test case lets a **hardcoded value** pass — configs and inputs vary across cases.
- ☐ Every failure/rejection case in the instruction is actually tested.
- ☐ **solve.sh** does exactly what the instruction says — no undocumented shortcuts, no skipped requirements.
- ☐ Any choice that changes the output is **pinned** (method, precedence, tie-breaks, schema) — or grading accepts all valid answers.
- ☐ Tolerances in the instruction **match** the verifier.
- ☐ Every test assertion traces back to a sentence in the instruction.
- ☐ Answers, goldens, and labels are not readable, editable, or inferable from the agent's side.
- ☐ Held-out cases require the general solution — not memorizable from the visible data.
- ☐ The verifier doesn't follow symlinks when staging agent files, and grades against a pristine, verifier-owned copy the agent never touched.
- ☐ The grader can't be spoofed by agent-controlled output text.
- ☐ The reward is written only by the verifier after grading — no agent-controlled code or lingering process can set it.

Get the instruction, tests, and solution describing the same contract — and keep the answer out of the agent's reach — and you've handled the large majority of what sends tasks back for revision.